INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

ENTREGA N.º 1 (25%)

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Alcance de la entrega del 25% 3
2. Descripción del proyecto 3
   2.1 Tecnologías 3
3. Arquitectura y entorno cloud 4
   3.1 Recursos del entorno (Terraform) 4
   3.2 Costo estimado 5
4. Endpoints definidos 5
   4.1 Formato único de respuesta 6
   4.2 Códigos HTTP 7
5. Registro y login diferenciados 8
   5.1 Pruebas automáticas del registro y login 11
6. Registro de incidentes 11
7. Conclusiones de la entrega 12
8. EXTRAS (adicionales a lo evaluado en el 25%) 13
   8.1 EXTRA: Diagrama de base de datos 13
   8.2 EXTRA: Pruebas de rendimiento y estrés con k6 (varias corridas) 14
   8.3 Análisis y cuello de botella encontrado 17
   8.4 EXTRA: Cifrado (ofuscación) del código 17
   8.5 EXTRA: Vista del programador 18
   8.6 EXTRA: Seguridad y calidad adelantadas 18
   8.7 EXTRA: Diseño de la aplicación móvil (Android e iOS) 19
9. Siguientes pasos (entrega del 80%) 23

1. Alcance de la entrega del 25%

Esta entrega presenta únicamente los puntos evaluados para la semana 9, según el documento «Pasos para presentación de entregables semana 9-11-13». Todo lo demás que ya se avanzó se presenta al final, en la sección 8, marcado como EXTRA, tal como pidió el docente.

Punto evaluado (25%)	Cómo se cumple	Evidencia en este informe	Estado
Entorno cloud preparado (cuenta AWS, VPC, EC2, RDS)	Infraestructura como código con Terraform: VPC con subredes públicas y privadas, EC2, RDS MySQL 8, Security Groups y SSM	Sección 3: diagrama, recursos y costo estimado	Definido como código (Terraform)
Endpoints definidos	Rutas en api/routes/api.php bajo /api/v1, formato JSON único y especificación OpenAPI	Sección 4	Cumplido
Registro y login diferenciados y funcionales	POST /auth/register y POST /auth/login con validaciones propias y pantallas separadas /registro y /login	Sección 5 (flujo, capturas y pruebas automáticas)	Cumplido

Repositorio: https://github.com/Josephml23/LeGrandHotel

2. Descripción del proyecto

LeGrandHotel es un sistema web de gestión de hospedaje. Cubre habitaciones y tipos, reservas y estadías (check-in y check-out), cochera, clientes, personal e inventario. Lo usan cinco tipos de usuario: Administrador, Recepcionista, Almacenero, Cliente y Programador. Además de la página web se diseñó una aplicación móvil para Android e iOS que usará la misma API (sección 8.7).

2.1 Tecnologías

Capa	Tecnología
Backend (API)	Laravel 12 · PHP 8.3 (Docker) · Sanctum (tokens) · Socialite (Google) · Pest (pruebas)
Frontend	React 18 · TypeScript · Vite · React Router · TanStack Query · Axios · Vitest
Base de datos y caché	MySQL 8 en Amazon RDS · Redis (ElastiCache, entrega 80%)
Contenedores	Docker + docker-compose (Nginx, PHP-FPM, MySQL, Redis en local)
Nube	AWS: VPC, EC2, RDS, S3, CloudFront, ALB, CloudWatch, SSM, ECR
Pruebas de carga	k6 con reporte HTML y gráficos
Repositorio	Monorepo: api/, web/, infra/, tests/k6/, docs/

3. Arquitectura y entorno cloud

El usuario solo llega a CloudFront (sitio web) y al balanceador (API). La API corre en EC2 y la base de datos no tiene acceso desde internet: vive en subredes privadas y solo acepta conexiones desde las instancias de la API.

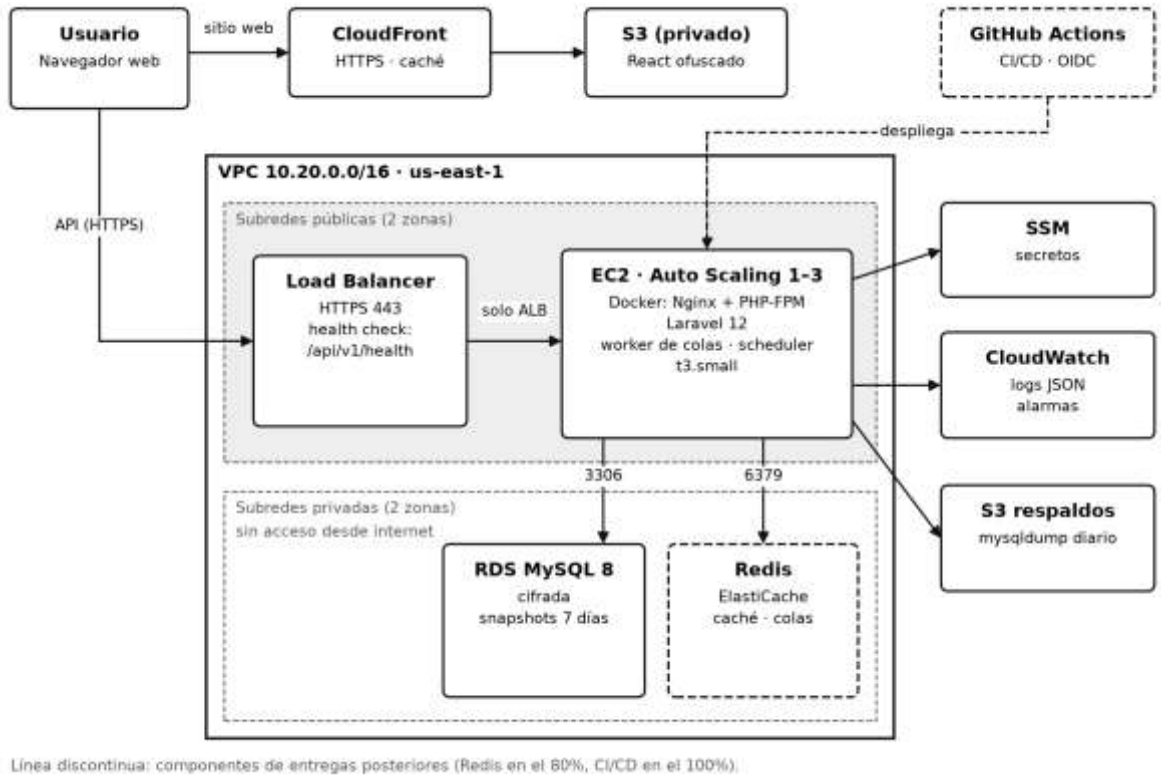


Figura 1. Arquitectura de LeGrandHotel en AWS (us-east-1).

3.1 Recursos del entorno (Terraform)

Todo el entorno está descrito como código en infra/terraform, así se puede crear, revisar y destruir de forma repetible y sin pasos manuales.

Recurso	Configuración	Archivo
VPC	10.20.0.0/16, 2 subredes públicas y 2 privadas en 2 zonas de disponibilidad	network.tf
EC2	t3.small con Docker (Nginx + PHP-FPM), Auto Scaling de 1 a 3 instancias	compute.tf
RDS	MySQL 8, db.t3.micro, 20 GB gp3 cifrado, subred privada, snapshots automáticos de 7 días	data.tf
Security Groups	ALB: 443/80 · EC2: solo desde el ALB · RDS: 3306 solo desde EC2	network.tf
SSM Parameter Store	Secretos y variables de producción (contraseña de BD como SecureString)	data.tf
Presupuesto	Alerta de USD 10 desde el primer día	monitoring.tf

Fragmento 1. *infra/terraform/data.tf y network.tf (resumen)*

```
resource "aws_db_instance" "main" {
  engine                  = "mysql"
  engine_version          = "8.0"
  instance_class          = var.db_instance_class   # db.t3.micro
  storage_encrypted       = true
  publicly_accessible     = false                   # sin IP pública
  db_subnet_group_name    = aws_db_subnet_group.main.name   # subredes privadas
  vpc_security_group_ids  = [aws_security_group.db.id]
  backup_retention_period = 7                       # snapshots diarios, 7 días
}

resource "aws_security_group" "db" {
  ingress {
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]    # solo desde las EC2 de la API
  }
}
```

Por qué se usó: la base de datos es lo más sensible del sistema. Con publicly_accessible = false y las subredes privadas no tiene dirección en internet; el Security Group solo deja entrar al puerto 3306 desde las instancias de la API. Además se cifra el disco y quedan 7 días de copias automáticas. Al estar escrito en Terraform, el mismo entorno se puede crear o destruir con un comando y revisar antes de aplicar.

3.2 Costo estimado

Recurso	USD/mes (24/7)
EC2 t3.small	≈ 15
RDS db.t3.micro + 20 GB	≈ 15
IPv4 públicas	≈ 4–8
Total 25% (sin ALB ni Redis)	≈ 35; ≈ 12 apagando EC2 y RDS fuera de horario

4. Endpoints definidos

Todas las rutas viven bajo /api/v1 y responden JSON. Salvo autenticación y salud, exigen un token de Sanctum. Los listados aceptan ?page= y ?per_page= (máximo 50). La especificación completa está en docs/endpoints/openapi.yaml y se puede abrir en editor.swagger.io.

Módulo	Método y ruta	Acceso
Salud	GET /health	Público
Autenticación	POST /auth/register	Público (3/min)
Autenticación	POST /auth/login	Público (5/min)
Autenticación	GET /auth/me · POST /auth/logout · POST /auth/refresh	Autenticado
Autenticación	GET /auth/google/redirect · GET /auth/google/callback	Público
Tipos de habitación	GET, POST /room-types · GET, PUT, DELETE /room-types/{id}	Admin (escritura), Recepción (lectura)

Módulo	Método y ruta	Acceso
Habitaciones	GET, POST /rooms · GET, PUT, DELETE /rooms/{id} · PATCH /rooms/{id}/status	Admin, Recepción
Habitaciones	GET /rooms/available?from=&to=	Admin, Recepción, Cliente
Clientes	CRUD /clients · GET, PUT /me/client	Admin, Recepción · Cliente
Reservas y estadías	CRUD /reservations · POST /reservations/{id}/check-in · POST /stays/{id}/check-out	Admin, Recepción, Cliente (solo las suyas)
Cochera	CRUD /parking-spots · CRUD /parking-assignments	Admin, Recepción
Inventario	CRUD /inventory-items · POST /inventory-items/{id}/movements	Admin, Almacenero
Personal y usuarios	CRUD /staff · CRUD /users · PUT /users/{id}/roles	Admin
Reportes	GET /reports/occupancy · GET /reports/income	Admin
Programador	GET /dev/overview · /dev/routes · /dev/requests · /dev/errors	Programador, Admin

Fragmento 2. *api/routes/api.php (extracto)*

```
Route::get('health', HealthController::class);

Route::prefix('auth')->group(function () {
    Route::post('register', [AuthController::class, 'register'])
        ->middleware('throttle:register');
    Route::post('login', [AuthController::class, 'login'])
        ->middleware('throttle:login');
});

Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
    Route::get('auth/me', [AuthController::class, 'me']);
    Route::middleware('role:admin')->group(function () {
        Route::post('room-types', [RoomTypeController::class, 'store']);
        // ...
    });
});
```

Por qué se usó: todas las rutas se declaran en un solo archivo, agrupadas por quién puede usarlas. Las públicas (registro y login) llevan su propio límite de intentos; el resto exige un token de Sanctum y, cuando corresponde, un rol. Laravel agrega el prefijo /api/v1, así una versión futura de la API puede convivir con esta sin romper el frontend.

4.1 Formato único de respuesta

```
Éxito: { "success": true, "data": {...}, "meta": { "page": 1, "per_page": 15, "total": 120 } }
Error: { "success": false, "error": { "code": "ROOM_ALREADY_BOOKED", "message": "...", "details": {...} } }
```

Fragmento 3. *api/app/Support/ApiResponse.php (extracto)*

```
public static function success(mixed $data = null, int $status = 200,
                               array $meta = []): JsonResponse
{
    $body = ['success' => true, 'data' => $data];
    if ($meta !== []) { $body['meta'] = $meta; }
    return response()->json($body, $status);
}

public static function error(string $code, string $message, int $status,
                             mixed $details = null): JsonResponse
{
    $error = ['code' => $code, 'message' => $message];
    if ($details !== null) { $error['details'] = $details; }
    return response()->json(['success' => false, 'error' => $error], $status);
}
```

Por qué se usó: si cada controlador armara su respuesta a mano, el frontend tendría que adivinar el formato de cada endpoint. Con una sola clase todas las respuestas tienen la misma forma, y el frontend maneja los errores en un único lugar leyendo error.code.

4.2 Códigos HTTP

Código	Cuándo se devuelve
200 / 201 / 204	Lectura, creación y eliminación correctas
400	Cuerpo JSON mal formado
401	Sin token, token vencido o credenciales incorrectas
403	El rol no tiene permiso (aunque la ruta se escriba a mano)
404	Recurso o ruta inexistente (p. ej. ROOM_NOT_FOUND)
409	Conflicto: habitación ya reservada en esas fechas
422	Validación fallida (detalle por campo)
429	Demasiadas peticiones, con cabecera Retry-After
500 / 503	Error interno o servicio caído: mensaje claro y código de incidente, sin trazas

Fragmento 4. *api/app/Exceptions/ApiExceptionRenderer.php (extracto)*

```
return match (true) {
    $e instanceof ValidationException       => ApiResponse::error(..., 422),
    $e instanceof AuthenticationException   => ApiResponse::error(..., 401),
    $e instanceof AuthorizationException    => ApiResponse::error(..., 403),
    $e instanceof ModelNotFoundException    => $this->modelNotFound($e),      // 404
    $e instanceof ThrottleRequestsException => ApiResponse::error(..., 429),
    $this->isDatabaseDown($e)               => $this->serviceUnavailable($e), // 503
    default                                 => $this->internalError($e),      // 500
};
```

Por qué se usó: es el punto central que traduce cualquier error a un código HTTP y al formato único. Así ningún error sale como página HTML ni muestra trazas del servidor; en los 500 se entrega un código de incidente que también queda en el log, para encontrar el error exacto sin mostrar detalles internos al usuario.

5. Registro y login diferenciados

El registro y el inicio de sesión son dos endpoints, dos validaciones (FormRequest) y dos pantallas distintas:

	Registro	**Login**
Endpoint	POST /api/v1/auth/register	POST /api/v1/auth/login
Pantalla	/registro	/login
Datos	Nombre, correo, contraseña (2 veces), tipo y número de documento, teléfono, nacionalidad	Correo y contraseña
Validación	Correo y documento únicos; contraseña ≥ 8 con letras y números	Credenciales contra el hash bcrypt
Resultado	Crea la cuenta con rol Cliente y su ficha de huésped; envía correo de verificación; devuelve token	Devuelve token Bearer de 60 minutos y los roles del usuario
Errores	422 con detalle por campo; 429 al 4.º intento por minuto	401 credenciales incorrectas; 429 al 6.º intento por minuto

La contraseña nunca se guarda en texto plano: el modelo User usa el cast «hashed» (bcrypt, 12 rondas). Tras iniciar sesión, cada rol entra a su propio inicio: el personal al Panel, el Cliente a «Mis reservas» y el Programador a su vista de diagnóstico.



Figura 2. Pantalla de inicio de sesión (/login).

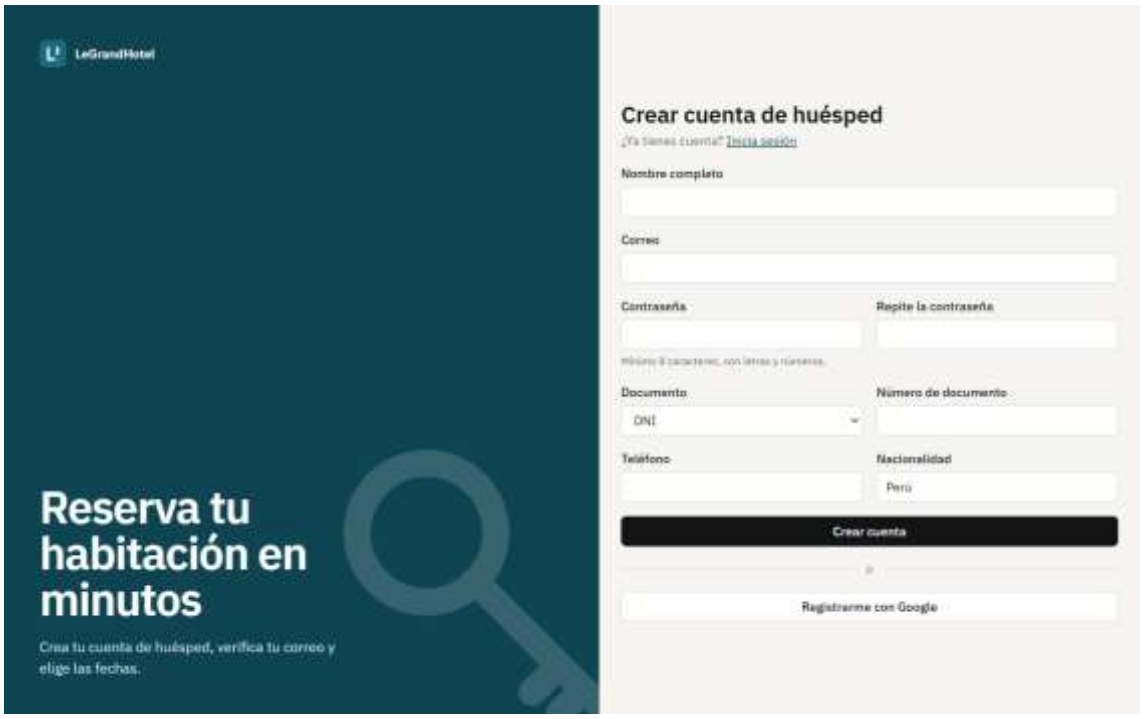


Figura 3. Pantalla de registro de huésped (/registro), separada del login.

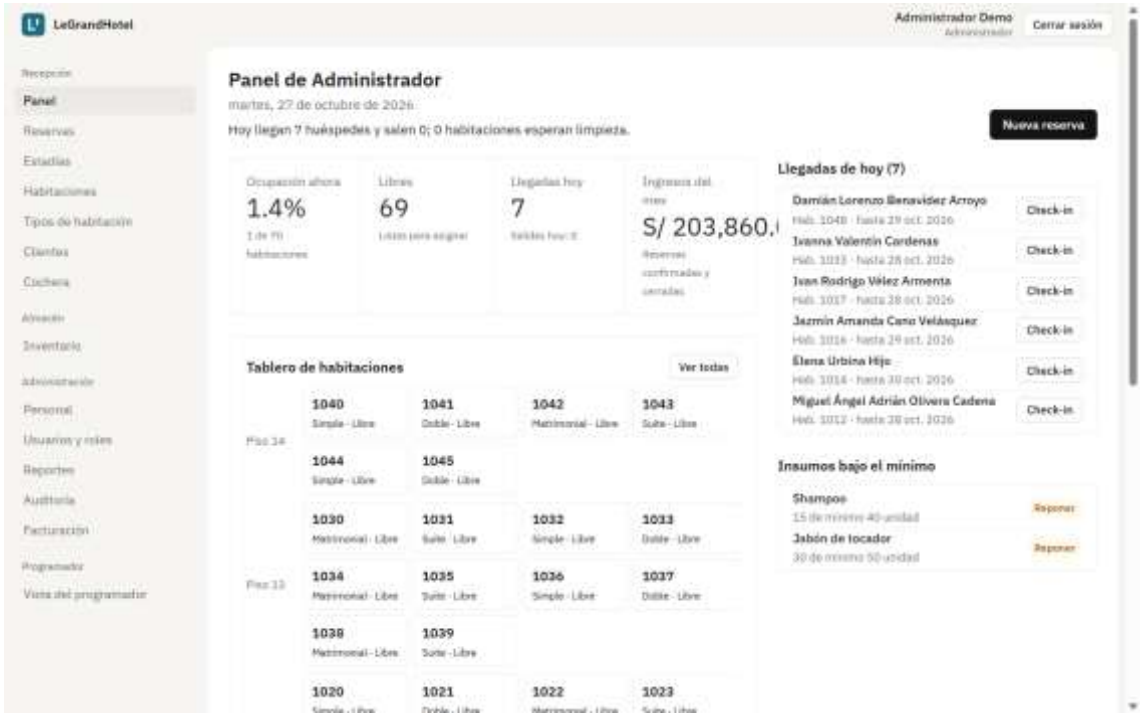


Figura 4. Panel de recepción después de iniciar sesión como Administrador.

Fragmento 5. *api/app/Http/Requests/Auth/RegisterRequest.php*

```
public function rules(): array
{
    return [
        'name'            => ['required', 'string', 'max:120'],
        'email'           => ['required', 'email', 'unique:users,email'],
        'password'        => ['required', 'confirmed',
                              Password::min(8)->letters()->numbers()],
        'document_type'   => ['required', Rule::in(Client::DOCUMENT_TYPES)],
        'document_number' => ['required', 'unique:clients,document_number'],
    ];
}
```

Por qué se usó: el registro tiene su propia validación, distinta de la del login. Se valida antes de llegar al controlador: si un dato falla, Laravel responde 422 con el mensaje de cada campo y el formulario lo muestra junto al campo. Las reglas unique evitan correos o documentos duplicados.

Fragmento 6. *api/app/Http/Controllers/Api/AuthController.php (extracto)*

```
public function register(RegisterRequest $request): JsonResponse
{
    $user = DB::transaction(function () use ($data) {
        $user = User::create([...]);              // la contraseña se guarda con bcrypt
        $user->assignRole(Role::CLIENT);           // todo registro público es Cliente
        Client::create(['user_id' => $user->id, ...]);
        return $user;
    });
    event(new Registered($user));                  // envía el correo de verificación
    return ApiResponse::created($this->tokenPayload($user, 'web'));
}

public function login(LoginRequest $request): JsonResponse
{
    $user = User::where('email', $request->string('email'))->first();
    if (! $user || ! Hash::check($request->password, $user->password)) {
        throw new ApiException('INVALID_CREDENTIALS', '...', 401);
    }
    return ApiResponse::success($this->tokenPayload($user, ...));
}
```

Por qué se usó: son dos acciones separadas con responsabilidades distintas. El registro crea en una transacción la cuenta, su rol y su ficha de huésped: si algo falla, no queda nada a medias. El login solo compara la contraseña con el hash bcrypt y responde el mismo mensaje si falla el correo o la contraseña, para no revelar qué correos existen. Ambos devuelven un token Bearer de Sanctum que vence a los 60 minutos.

Fragmento 7. *api/app/Models/User.php y AppServiceProvider.php (extractos)*

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password'          => 'hashed',   // bcrypt automático
    ];
}

RateLimiter::for('login', fn (Request $r) => Limit::perMinute(5)
    ->by(Str::lower($r->input('email')).'|'.$r->ip()));
RateLimiter::for('register', fn (Request $r) => Limit::perMinute(3)->by($r->ip()));
```

Por qué se usó: con el cast hashed, cualquier contraseña que se asigne al usuario se cifra con bcrypt antes de guardarse, sin depender de que cada desarrollador lo recuerde. Los límites de intentos frenan ataques de fuerza bruta: el login permite 5 intentos por minuto por correo e IP y el registro 3 por minuto por IP; al superarlos la API responde 429.

Fragmento 8. *web/src/App.tsx (extracto)*

```
<Route element={<GuestRoute />}>
  <Route path="/login" element={<LoginPage />} />
  <Route path="/registro" element={<RegisterPage />} />
</Route>

<Route element={<ProtectedRoute />}>
  <Route element={<ProtectedRoute roles={['admin']} />}>
    <Route path="/personal" element={<StaffPage />} />
  </Route>
</Route>
```

Por qué se usó: en el frontend el login y el registro son dos pantallas con su propia ruta. GuestRoute las oculta si ya hay sesión, y ProtectedRoute comprueba la sesión y el rol: si alguien escribe a mano una ruta que su rol no puede ver, se muestra la pantalla 403. El backend valida lo mismo en cada endpoint, así que esconder la pantalla no es la única protección.

5.1 Pruebas automáticas del registro y login

Prueba (api/tests/Feature/AuthTest.php)	Resultado
Registro crea un Cliente con contraseña bcrypt y devuelve token	Pasa
Registro con datos inválidos responde 422	Pasa
No se permite registrar un correo repetido	Pasa
Registro limitado a 3 por minuto (429 con Retry-After)	Pasa
Login con credenciales válidas devuelve token y roles	Pasa
Login con contraseña incorrecta responde 401	Pasa
Login limitado a 5 intentos por minuto	Pasa
Sin token o con token vencido responde 401	Pasa
Frontend: login y registro son pantallas separadas; errores 401 y 422 visibles (Vitest)	Pasa

Resultado global: 89 pruebas de la API (376 aserciones) y 14 pruebas del frontend pasan. El análisis estático con PHPStan no reporta errores y el pipeline de GitHub Actions está en verde.

6. Registro de incidentes

Toda caída, aunque sea en desarrollo, se registra el mismo día (docs/incidentes.md).

Fecha	Qué falló	Motivo	Solución	Tiempo
26/10/2026	El login devolvía 500 con código de incidente XG1R7UASWT	«php artisan serve» lanzaba PHP sin la extensión pdo_sqlite	Servidor local con php -S cargando la extensión	≈ 2 min

Fecha	Qué falló	Motivo	Solución	Tiempo
26/10/2026	El frontend mostró la pantalla 503	El servidor apuntaba a un index.php inexistente	Usar api/public/index.php como router; la pantalla 503 reintentó sola	≈ 1 min
27/10/2026	Primera corrida de k6: 100% de peticiones fallidas	PHP escuchaba solo en IPv6 (::1) y k6 resolvía localhost a IPv4	BASE_URL=http://[::1]:8000; se repitieron las corridas	≈ 5 min

7. Conclusiones de la entrega

- Los tres puntos del 25% están implementados: endpoints definidos y documentados, registro y login separados y probados, y el entorno cloud descrito como código.
- Las pruebas de carga locales (sección 8.2) muestran que el servidor de desarrollo se satura con pocos usuarios. En la entrega del 80% se repetirán en AWS con Nginx + PHP-FPM, que atiende varias peticiones a la vez.

8. EXTRAS (adicionales a lo evaluado en el 25%)

Todo lo de esta sección es EXTRA: no forma parte de lo evaluado en el 25% y se presenta como avance adicional.

8.1 EXTRA: Diagrama de base de datos

13 tablas del dominio en MySQL 8 con los índices que pide el plan: rooms(status), reservations(check_in, check_out), reservations(client_id), clients(document_number) único e inventory_items(name). La versión editable está en DBML (docs/diagramas/legrandhotel.dbml) y se abre en dbdiagram.io.

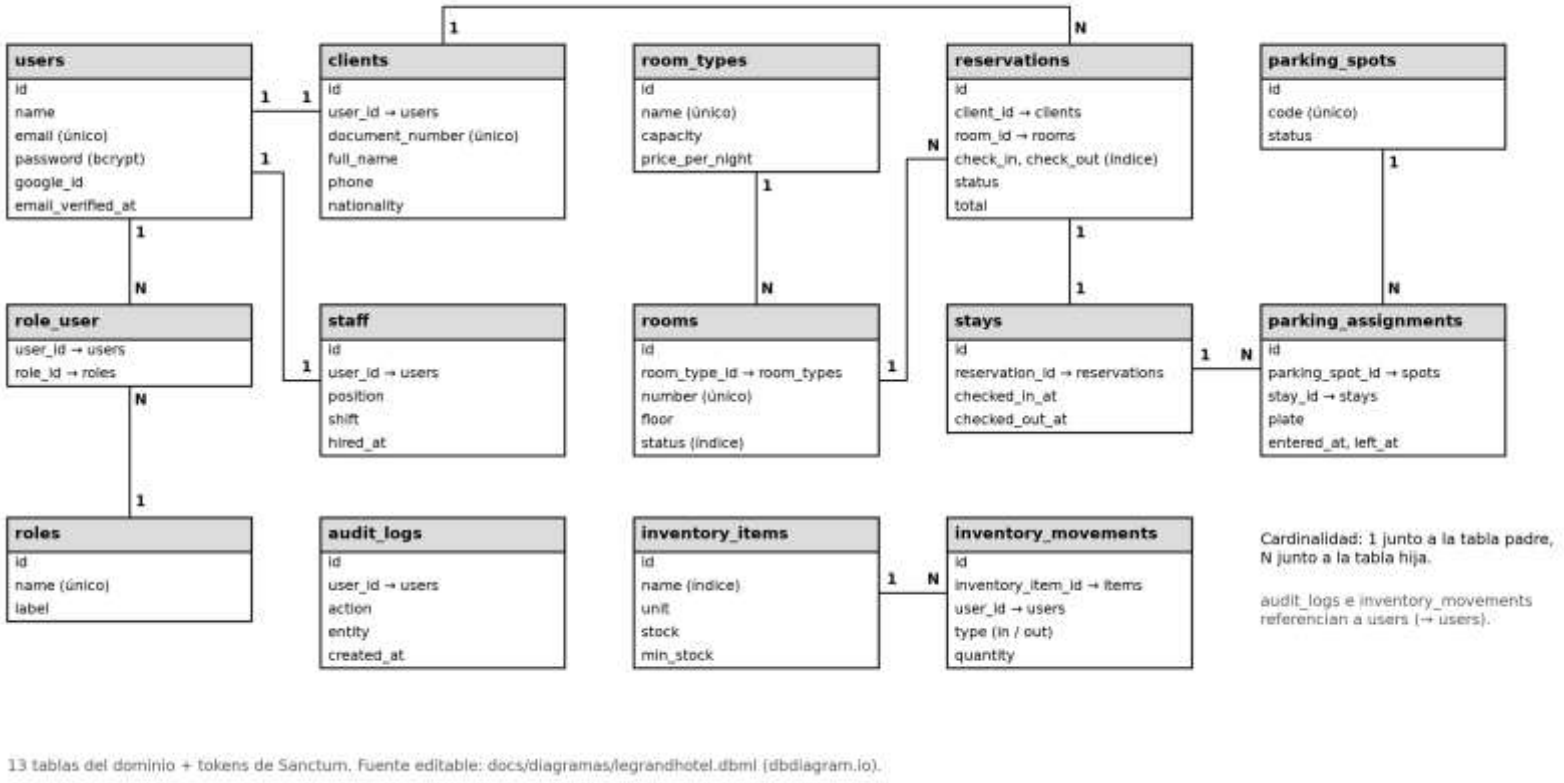


Figura 5. Modelo entidad-relación de LeGrandHotel.

8.2 EXTRA: Pruebas de rendimiento y estrés con k6 (varias corridas)

Se hicieron cinco corridas con k6 contra la API en el equipo de desarrollo (Windows, PHP 8.2, servidor integrado de PHP que atiende una petición a la vez, SQLite y caché en archivo). No es el servidor de producción: sirven para comparar la carga entre corridas y detectar cuellos de botella antes de subir a AWS.

Escenario: 70% lecturas (habitaciones disponibles, mis reservas, tipos de habitación), 20% login y 10% creación de reservas. Cada escalón sube en 10 s, mantiene la carga 45 s y baja en 5 s. Meta del plan: p95 menor a 500 ms y 0% de errores.

Corrida	Peticiones	Errores	p50	p95	Resultado
Smoke (5 VU, 30 s): salud, BD, caché y 401	222	0.00%	203 ms	321 ms	Cumple (100% de verificaciones)

VU	Peticiones	Pet./s	p50	p95	Máx.	Errores	p95 < 500 ms
5	350	4.4	288 ms	528 ms	1,804 ms	0.00%	No cumple
10	413	5.2	910 ms	1,309 ms	1,598 ms	0.00%	No cumple
25	438	5.2	3,361 ms	3,952 ms	4,197 ms	0.00%	No cumple
50	483	5.3	7,016 ms	7,885 ms	8,195 ms	0.00%	No cumple

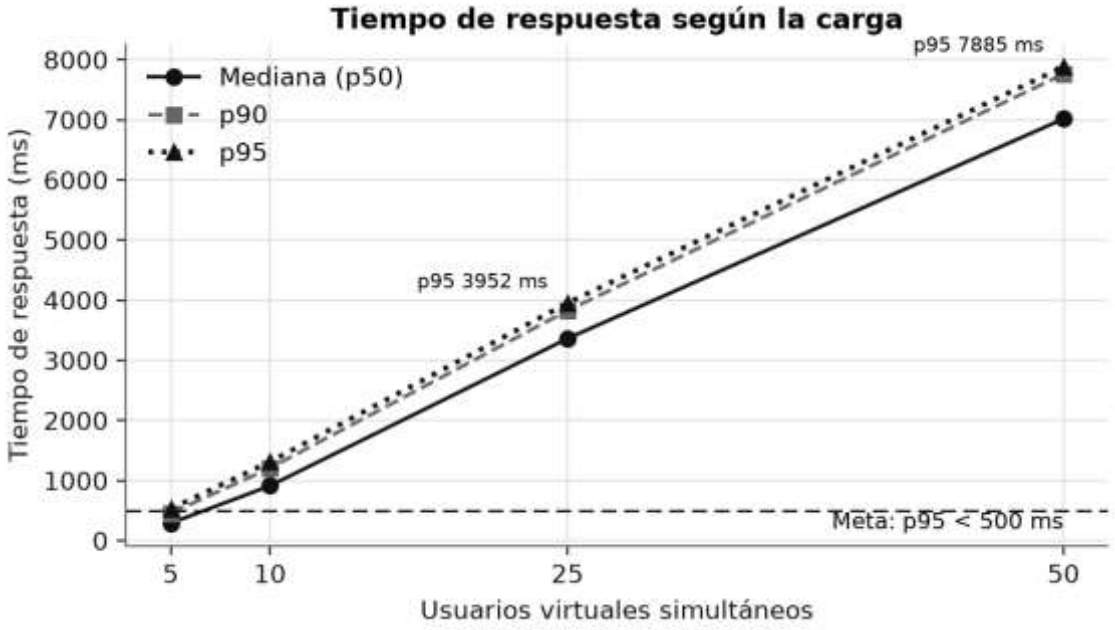


Figura 6. Tiempo de respuesta (p50, p90, p95) según la cantidad de usuarios virtuales.

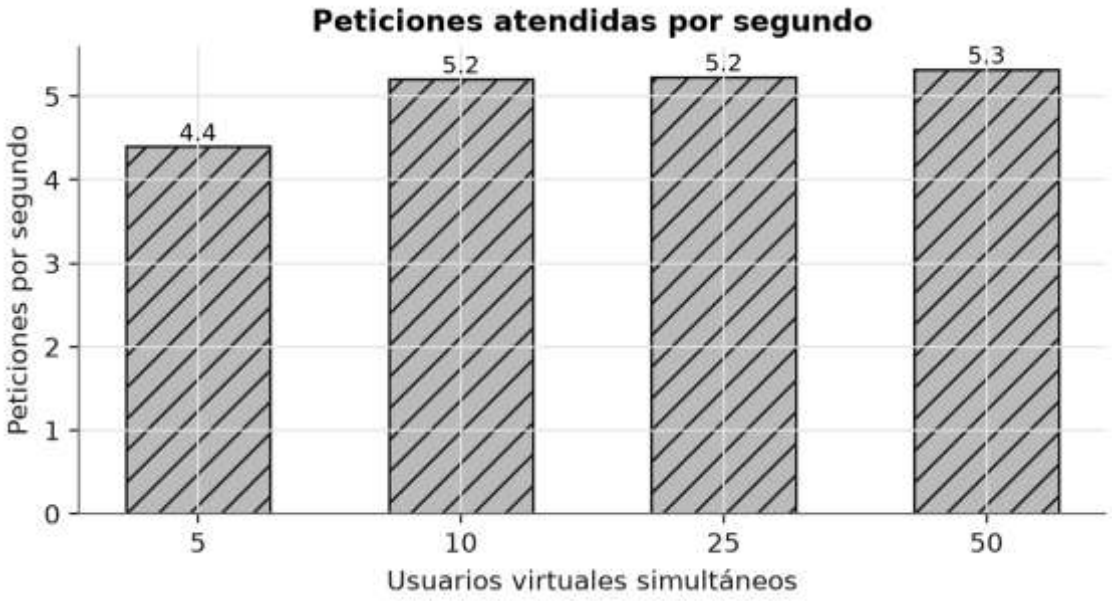


Figura 7. Peticiones atendidas por segundo en cada corrida.



Figura 8. Porcentaje de peticiones fallidas en cada corrida.

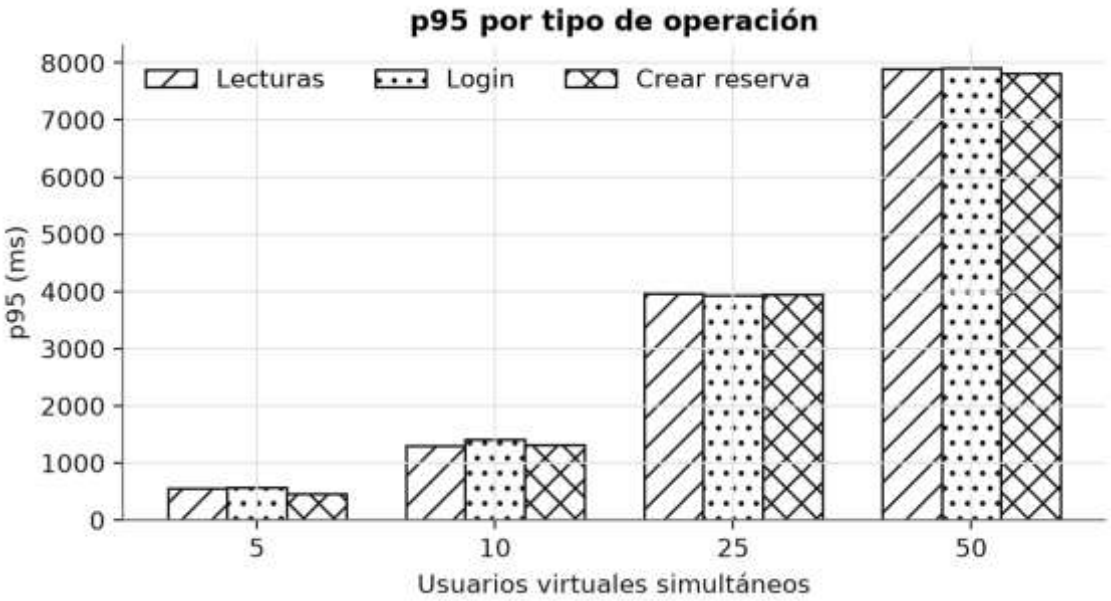


Figura 9. p95 por tipo de operación (lecturas, login, crear reserva).

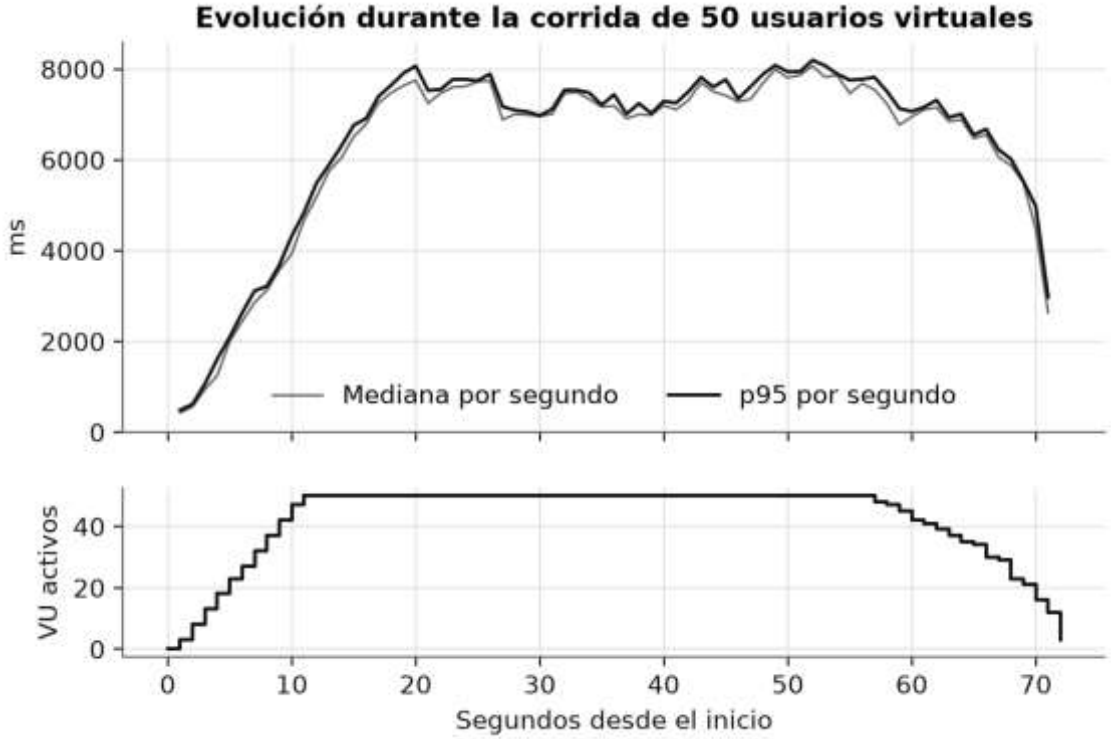


Figura 10. Evolución de la latencia segundo a segundo en la corrida de 50 usuarios.

Fragmento 9. *tests/k6/reservas.js (extracto)*

```
escalon: {
  stages: [
    { duration: '10s', target: VUS },   // sube hasta VUS usuarios
    { duration: '45s', target: VUS },   // mantiene la carga
    { duration: '5s',  target: 0 },     // baja
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],   // meta: p95 menor a 500 ms
    http_req_failed: ['rate<0.01'],     // menos de 1% de errores
  },
},
```

Por qué se usó: el mismo script se corre con distinta cantidad de usuarios (VUS) para comparar corridas entre sí. Los thresholds convierten la meta del plan (p95 menor a 500 ms y menos de 1% de errores) en una regla: si no se cumple, k6 marca la prueba como fallida sin interpretar los números a mano.

8.3 Análisis y cuello de botella encontrado

- Con 5 usuarios el p95 fue 528 ms; con 50 usuarios llegó a 7,885 ms. Las peticiones por segundo apenas suben (4.4 → 5.3): el servidor ya estaba al máximo de lo que puede atender.
- Causa: el servidor integrado de PHP en Windows procesa una petición a la vez. Cuando llegan más usuarios, las peticiones esperan en cola y el tiempo de respuesta crece de forma casi lineal, aunque el código de cada endpoint sea rápido (en la prueba masiva de Pest, el listado de 5000 reservas responde en menos de 500 ms).
- Con 50 VU, lecturas, login y creación de reservas tienen casi el mismo p95 (7,890, 7,905 y 7,820 ms). El tiempo lo domina la espera en la cola del servidor, no el trabajo de cada endpoint.
- A pesar de la saturación no hubo errores (0% en todas las corridas): la API responde más lento, pero no se cae ni pierde peticiones.
- Corrección prevista para el 80%: Docker con Nginx + PHP-FPM (20 procesos), OPcache, MySQL en RDS y Redis para caché y rate limit. Las mismas corridas se repetirán en AWS con 50–100 y luego 1000–5000 usuarios, para comparar el antes y el después.

Los reportes HTML interactivos de cada corrida (generados por k6) están en docs/pruebas/local-*.html y los datos crudos en docs/pruebas/datos/.

8.4 EXTRA: Cifrado (ofuscación) del código

El build de producción del frontend sale ofuscado con javascript-obfuscator: los nombres se reemplazan por identificadores hexadecimales y los textos se guardan codificados en base64. Comparación del mismo texto de la interfaz («No tienes acceso»):

Build	Comando	Veces que aparece legible
Normal	npm run build:plain	1
Ofuscado (producción)	npm run build	0

El backend no se publica: corre en un servidor privado sin acceso al código.

Fragmento 10. *web/vite.config.ts (extracto)*

```
renderChunk(code, chunk) {
  const result = JavaScriptObfuscator.obfuscate(code, {
    identifierNamesGenerator: 'hexadecimal',   // nombres ilegibles
    stringArray: true, stringArrayEncoding: ['base64'],   // textos codificados
    splitStrings: true, compact: true,
  })
  return { code: result.getObfuscatedCode(), map: null }
}
```

Por qué se usó: el código de una aplicación web se descarga al navegador, así que cualquiera puede leerlo. La ofuscación se aplica al paquete final de producción para dificultar copiarlo o estudiar su lógica, sin cambiar lo que hace. En desarrollo se trabaja con el código normal (npm run build:plain).

8.5 EXTRA: Vista del programador

El rol Programador tiene una pantalla de diagnóstico (/programador) que muestra, sin entrar al servidor: estado de la aplicación, la base de datos y la caché; versión y configuración (vida del token, rate limits, CORS); latencias p50/p95/p99 calculadas del log JSON; rutas más lentas; últimas peticiones; errores buscables por código de incidente, el mismo que ve el usuario en la pantalla 500; y la lista de todas las rutas de la API con su rol de acceso. No muestra contraseñas ni claves, y una prueba automática lo verifica.

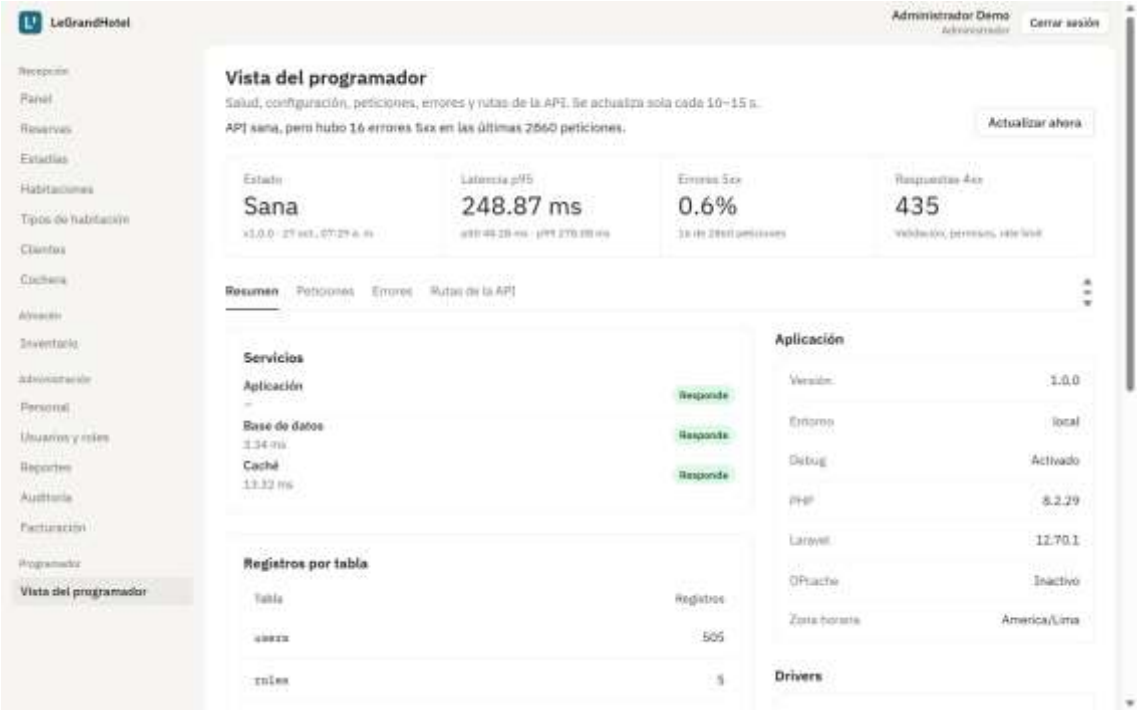


Figura 11. Vista del programador.

8.6 EXTRA: Seguridad y calidad adelantadas

- Rate limits: login 5/min, registro 3/min y API 60/min, con respuesta 429 y Retry-After (punto del 80%).

- Roles y permisos: el rol se valida antes de buscar el registro, así una ruta pedida a mano responde 403 sin revelar si el id existe.
- Pantallas de error en React: 404, «En construcción», 403, sesión expirada, 500 con código de incidente, 503 con reintento cada 30 s, sin conexión y 429 con cuenta regresiva.
- CI/CD en GitHub Actions: estilo (Pint), análisis (PHPStan), pruebas (Pest y Vitest), auditoría de dependencias y Semgrep en cada cambio. Auditoría actual: 0 vulnerabilidades en composer y npm y 0 hallazgos en Semgrep.
- Logs estructurados en JSON (una línea por petición con request_id, ruta, código y duración), listos para CloudWatch.

8.7 EXTRA: Diseño de la aplicación móvil (Android e iOS)

LeGrandHotel tendrá también una aplicación para teléfonos Android e iOS. En esta entrega se presenta su diseño: la tecnología, la arquitectura, las pantallas y el plan de construcción. El código de la app se desarrolla en las entregas del 80% y del 100%. La app no reemplaza a la web: cubre las tareas que conviene hacer en movimiento y usa la misma API, sin duplicar la lógica del negocio.

Usuario	Para qué usará la app
Cliente	Registrarse o entrar (también con Google), buscar disponibilidad, reservar, ver y cancelar sus reservas, recibir avisos
Recepcionista	Ver llegadas y salidas del día, hacer check-in y check-out desde el lobby, cambiar el estado de una habitación, registrar placas en la cochera
Almacenero	Registrar entradas y salidas de inventario en el almacén y ver los insumos bajo el mínimo
Administrador	Ver el panel resumido (ocupación, ingresos, alertas). La gestión completa sigue en la web

Tecnología elegida: React Native con Expo y TypeScript. Con un solo código se generan las dos aplicaciones y se reutiliza lo que el equipo ya usa en la web: React, TypeScript, TanStack Query, Axios y los mismos tipos de datos.

Opción	A favor	En contra	Decisión
React Native + Expo	Un código para Android e iOS; mismo lenguaje y librerías que la web; compila iOS en la nube sin Mac (EAS Build)	Funciones nativas muy específicas requieren módulos propios	Elegida
Flutter	Buen rendimiento y diseño uniforme	Otro lenguaje (Dart): no se reutiliza nada de la web	Descartada
Nativo (Kotlin + Swift)	Acceso total al sistema	Dos aplicaciones distintas: el doble de trabajo para 4 personas	Descartada
Web instalable (PWA)	Sin código nuevo	Notificaciones y almacenamiento seguro	Complemento

Opción	A favor	En contra	Decisión
		limitados en iOS; no está en las tiendas	

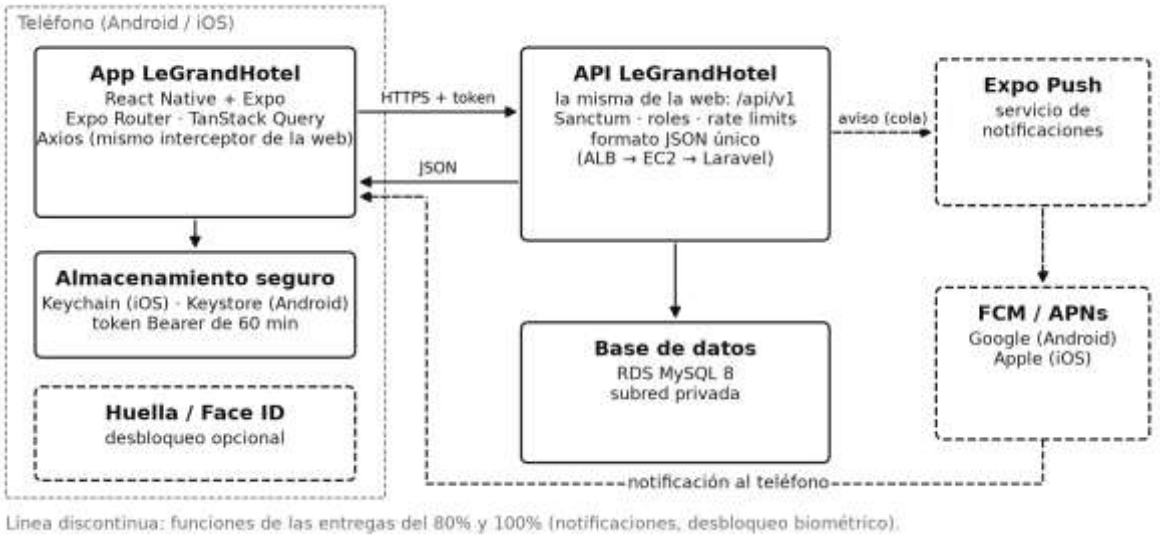


Figura 12. Arquitectura de la aplicación móvil: misma API y notificaciones por FCM (Android) y APNs (iOS).

Necesidad	En la app	Equivalente en la web
Navegación	Expo Router (barra de pestañas por rol)	React Router
Llamadas a la API y caché	TanStack Query + Axios con el mismo interceptor	Igual
Guardar la sesión	expo-secure-store: Keychain (iOS) y Keystore (Android)	sessionStorage
Login con Google	expo-auth-session y enlace legrandhotel://auth/callback	Socialite con redirección
Notificaciones	expo-notifications → FCM / APNs	—
Desbloqueo	Huella o Face ID (expo-local-authentication)	—
Pruebas	Jest + Testing Library; Maestro para flujos completos	Vitest; Playwright

Cambios que necesitará la API (pocos, porque la app usa los mismos endpoints):

- Registrar el token de notificaciones de cada teléfono (POST /devices) y enviar avisos en cola: reserva confirmada, recordatorio de check-in e insumo bajo el mínimo.
- Login con Google que vuelva a la app (enlace profundo) además de a la web.
- Cabecera X-App-Version para pedir que se actualice la app si cambia la API (respuesta 426).
- Nada más: el login ya acepta un nombre de dispositivo para que cada teléfono tenga su propio token; CORS no aplica a apps nativas; roles y límites de intentos son los mismos.

Seguridad: el token se guarda en el almacenamiento seguro del sistema y nunca en texto plano. Se renueva solo antes de vencer, con el mismo POST /auth/refresh de la web. El

personal puede activar el desbloqueo con huella o rostro. La app solo usa HTTPS y no contiene secretos: todo secreto queda en el servidor.

Entrega	Alcance de la app
25% (esta)	Diseño: tecnología, arquitectura, pantallas por rol y bocetos
80%	Proyecto Expo en mobile/; login y registro; flujo del cliente (reservar, mis reservas); recepción «Hoy» con check-in y check-out; notificaciones; versión de prueba para Android
100%	Inventario, cochera y panel del administrador; versión de prueba para iOS en TestFlight; pruebas de flujos completos con Maestro; publicación de prueba en Google Play y App Store

Publicación: las versiones de prueba se distribuyen con Google Play (prueba interna) y Apple TestFlight. Para publicar se necesitan la cuenta de Google Play Console (pago único de USD 25) y la del Apple Developer Program (USD 99 al año). El diseño completo está en docs/movil/diseno-app-movil.md.

Bocetos de la aplicación móvil

Pantallas principales en escala de grises. Usan los mismos datos y reglas que la web; la barra inferior cambia según el rol del usuario.



Figura 13. Bocetos: login, inicio y reserva del cliente; «Hoy» y tablero de habitaciones de recepción.

9. Siguientes pasos (entrega del 80%)

- Desplegar con el pipeline: imagen Docker en ECR, frontend en S3 + CloudFront, ALB con HTTPS.
- Repetir las pruebas k6 en AWS con 50–100 y 1000–5000 usuarios, con reportes HTML y comparación antes/después.
- Configurar el login con Google (credenciales OAuth) y los respaldos automáticos de RDS y S3.
- Aplicación móvil: crear el proyecto Expo, construir el login y el flujo del cliente y la recepción, activar las notificaciones y generar la primera versión de prueba para Android.